GROUP PROJECT TECHNICAL REPORT

AI Learning Platform with Intelligent Tutor & AI Command Terminal

Course: IT41073 - Service Oriented Computing
Project Title: IntelliLearn — Microservices LMS, RAG AI Tutor & AI Command Terminal
Author / Contributor: Chamod (30% Subsystem Lead & Microservices Architect)
Branch: `chamodDev`
Date: August 11, 2026

---

1. Executive Summary & Project Introduction

Modern online education requires scalable, resilient, and intelligent software platforms. The **IntelliLearn Platform** is a cloud-native Learning Management System (LMS) designed using a **Microservices Architecture** paired with a **Project-Aware & Web-Aware Intelligent AI Tutor Chatbot** and an **AI Command Terminal** for DevOps/Linux assistance.

Traditional monolithic LMS architectures suffer from single points of failure, scaling bottlenecks during exam peaks, and tightly coupled database schemas. In contrast, IntelliLearn decouples business logic into independent microservices (`user-service`, `course-service`, `quiz-service`, `progress-service`, `tutor-service`) fronted by a centralized **API Gateway** on port `8080`.

Students can enroll in courses, attempt practice assessments with automatic score feedback, track module progress percentages, query a **Project-Aware & Web-Aware AI Chatbot** grounded directly in uploaded course documents and IntelliTutor's microservices architecture, and utilize an **AI Command Terminal** to convert natural language into safe, explained CLI shell commands with automated error diagnosis and controlled process execution.

---

2. System Architecture & High-Level Design

2.1 Microservices Architecture Diagram

```
graph TD
    Client["React Frontend Client\n(Port 3000 / Nginx)\n• Dashboard & AI Chatbot Component & AI Terminal"]

    subgraph Edge Layer
        Gateway["API Gateway\n(Port 8080)\n• Centralized Routing\n• Keycloak OAuth2 / JWT Validation\n• Redis IP Rate Limiting (/api/tutor/chat, /api/tutor/command/**)\n• Global CORS Policy"]
    end

    subgraph Microservices Layer
```

```
        UserService["User Management Service\n(Port 8081)\n• User Auth &
Profiles\n• MongoDB userdb"]
        CourseService["Course Management Service\n(Port 8082)\n• Course Syllabus &
Enrolment\n• MongoDB coursedb"]
        QuizService["Quiz & Assessment Service\n(Port 8083)\n• Quiz Player &
Evaluation\n• MongoDB quizdb (quiz_attempts)"]
        ProgressService["Learning Progress Service\n(Port 8084)\n• Progress
Analytics & Percentages\n• MongoDB progressdb"]
        TutorService["AI Tutor & AI Command Subsystem\n(Port 8085)\n• Question
Router (Categories A-E)\n• Project RAG Knowledge Base\n• Server-Side Web Search
Abstraction\n• Project Troubleshooter Engine (401, CORS, DB)\n• Conversation Memory
(tutordb.chat_sessions)\n• LLM Abstraction (Gemini/Mock)\n• Command Safety &
Controlled Execution Engine"]
    end

    subgraph Shared Infrastructure Layer
        MongoDB[(MongoDB Database 7.0\nPersistent Volume: mongodb-data)]
        Redis[(Redis Cache 7.0\nRate Limiting Key-Value)]
        Keycloak[(Keycloak IAM 25.0\nOAuth2 & JWT Issuer)]
    end

    Client -->|HTTPS / REST API| Gateway
    Gateway -->|Redis Rate Limiter| Redis
    Gateway -->|Validate JWT Token| Keycloak
    Gateway -->|Route /api/users, /api/auth| UserService
    Gateway -->|Route /api/courses| CourseService
    Gateway -->|Route /api/quizzes| QuizService
    Gateway -->|Route /api/progress| ProgressService
    Gateway -->|Route /api/tutor/**| TutorService

    UserService --> MongoDB
    CourseService --> MongoDB
    QuizService --> MongoDB
    ProgressService --> MongoDB
    TutorService --> MongoDB
```

---

3. Microservices Detailed Specifications

3.1 User Management Service (`user-service`)

- **Port**: `8081` | **Database**: MongoDB (`userdb`)
- **Endpoints**: `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/{id}`

3.2 Course Management Service (`course-service`)

- **Port**: `8082` | **Database**: MongoDB (`coursedb`)
- **Endpoints**: `GET /api/courses`, `GET /api/courses/{id}`, `POST /api/courses/{id}/enroll`, `POST /api/courses`

3.3 Quiz & Assessment Service (`quiz-service`)

- **Port**: `8083` | **Database**: MongoDB (`quizdb`, collection `quiz_attempts`)
- **Endpoints**: `GET /api/quizzes`, `POST /api/quizzes/{id}/submit`, `GET /api/quizzes/{quizId}/attempts/{userId}`, `POST /api/quizzes`

3.4 Learning Progress Service (`progress-service`)

- **Port**: `8084` | **Database**: MongoDB (`progressdb`)
- **Endpoints**: `GET /api/progress`, `GET /api/progress/{userId}`, `POST /api/progress`, `PUT /api/progress/{userId}/{courseId}`

3.5 AI Tutor & AI Command Terminal Subsystem (`tutor-service`)

- **Port**: `8085` | **Database**: MongoDB (`tutordb`, collections `chat_sessions`, `tutor_interactions`, `command_history`)
- **Endpoints**:
  - `POST /api/tutor/chat`: Project-Aware & Web-Aware Intelligent AI Chatbot.
  - `GET /api/tutor/chat/session/{sessionId}`: Fetch chat session history.
  - `DELETE /api/tutor/chat/session/{sessionId}`: Clear chat session.
  - `POST /api/tutor/command/generate`: Translates natural language into CLI commands.
  - `POST /api/tutor/command/explain`: Line-by-line tokenized CLI flag deconstruction.
  - `POST /api/tutor/command/execute`: Controlled safe process execution sandbox with 5-second timeout.
  - `POST /api/tutor/command/diagnose`: Diagnoses terminal error logs and suggests diagnostic commands.

---

4. Empirical Test & Verification Matrix

Verification Test	Method / Endpoint	Expected Result	Status
Gateway Health	`GET /gateway/health`	`200 OK {"status":"Gateway is running"}`	**PASS**
Project-Specific Question	`POST /api/tutor/chat`	`200 OK "category":"PROJECT_SPECIFIC",`	**PASS**

Verification Test	Method / Endpoint	Expected Result	Status
		`sourceType:"PROJECT"`	
Project Troubleshooting	`POST /api/tutor/chat` ("401 error")	`200 OK` `"category":"PROJECT_TROUBLESHOOTING"`, diagnosis & checks	**PASS**
Web Search Integration	`POST /api/tutor/chat` ("Spring Boot version")	`200 OK` `"webSearchUsed": true`, web citations returned	**PASS**
Chat Session Memory	`POST /api/tutor/chat` (2 turns)	`200 OK` Context maintained across turns in MongoDB	**PASS**
Controlled Safe Execution	`POST /api/tutor/command/execute`	`200 OK` `status:"SUCCESS"`, `exitCode:0`, `stdout`	**PASS**
Dangerous Command Safety Lock	`POST /api/tutor/command/execute` (`rm -rf`)	`200 OK` `status:"BLOCKED_HIGH_RISK"`	**PASS**

---

5. Individual Contribution Matrix (Target: ~30%)

Subsystem / Feature	Contribution %	Author	Key Technical Work
Project-Aware AI Chatbot Subsystem	100%	Chamod	Built `ChatOrchestratorService`, `QuestionRouterService`, `ProjectKnowledgeService`, & `ChatController`
Server-Side Web Search Integration	100%	Chamod	Created `WebSearchService.java` for safe external documentation fetching without exposing API keys

Subsystem / Feature	Contribution %	Author	Key Technical Work
Multi-Turn Conversation Memory	100%	Chamod	Built `ChatSession` document, repository, & `ConversationMemoryService` in MongoDB `tutordb`
Project Troubleshooter Engine	100%	Chamod	Created diagnostic reasoning engine (`ProjectTroubleshooterService.java`) for 401, CORS, & DB errors
AI Command Terminal (Phase 1 & 2)	100%	Chamod	Built CLI generator, flag deconstructor, controlled execution runner, and React UI components
Total Contribution Weight	**~30%**	**Chamod**	**Architected & Implemented AI Assistance & Command Subsystem**